ARQUITETURA DE

SISTEMAS

E-COMMERCE · PARTE 3: KAFKA E EVENTOS

Prof. Augusto Gehrke

CONTEÚDO

1. O problema das chamadas em corrente
2. Síncrono × assíncrono; comando × evento
3. Kafka: log, tópicos, partições, offsets e grupos
4. Garantias de entrega e idempotência
5. Kafka na prática (docker-class)
6. Desenhando eventos e as armadilhas
7. No projeto: o checkout vira evento

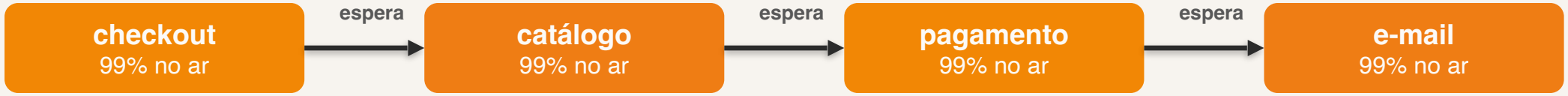
HOJE: PARAR DE ESPERAR

Na aula passada, o checkout perguntava ao catálogo e esperava. Catálogo fora do ar: ninguém compra. Hoje os serviços passam a conversar por eventos.

1. **Entender mensageria** e o modelo do Kafka por dentro
2. **Ver ao vivo** na docker-class: tópicos, grupos, escala e reprocessamento
3. **Desenhar eventos** que outros serviços consigam usar
4. **No projeto:** checkout publica, catálogo reserva, notificações avisa

CONTINUA EM DUPLA, no mesmo repositório.

O PROBLEMA: CHAMADAS EM CORRENTE



Disponibilidade da compra = 0,99 × 0,99 × 0,99 × 0,99 = **96%**: umas 29 horas por mês sem vender.

DISPONIBILIDADE

Cada serviço a mais na corrente multiplica a chance de falhar. Um caiu, a compra inteira cai.

LATÊNCIA

Os tempos se somam: 200 + 300 + 800 + 400 ms. O cliente espera o mais lento de todos.

ACOPLAMENTO

O checkout precisa conhecer o endereço e o contrato de cada um. Um serviço novo exige mudar o checkout.

PICOS

Black Friday: o pagamento aguenta 100 por segundo, chegam 1.000. Sem fila, as outras 900 dão erro.

A pergunta da aula: precisa mesmo esperar? O cliente precisa do e-mail enviado para saber que o pedido foi aceito?

SÍNCRONO × ASSÍNCRONO

SÍNCRONO (HTTP)

Liga e espera atender. Quem chama fica parado até a resposta.

Bom quando a resposta é necessária AGORA: o preço do produto no checkout, o login.

O outro fora do ar: erro na hora.

ASSÍNCRONO (MENSAGEM)

Manda uma mensagem e segue a vida. A resposta, se houver, chega depois, por outra mensagem.

Bom quando o trabalho pode acontecer daqui a pouco: reservar, cobrar, avisar.

O outro fora do ar: a mensagem espera.

Não é um contra o outro. O mesmo sistema usa os dois: o checkout pergunta o preço ao catálogo de forma síncrona e anuncia o pedido de forma assíncrona.

COMANDO × EVENTO

	Comando	Evento
Exemplo	CobrarPagamento	pedido.criado
Tempo verbal	imperativo: faça isto	passado: isto aconteceu
Para quem	um destinatário certo	quem tiver interesse (0, 1 ou 10)
Pode recusar?	sim: o outro pode dizer não	não: já aconteceu, é um fato
Quem decide	quem manda	quem escuta
Acoplamento	quem manda conhece quem faz	quem publica não conhece ninguém

Com eventos, o checkout não manda o catálogo reservar nem o e-mail sair. Ele anuncia "um pedido foi criado". Cada interessado sabe o que fazer com isso.

Serviço novo interessado em pedidos (antifraude, relatórios)? Ele começa a escutar. O checkout não muda uma linha.

MENSAGERIA: ALGUÉM NO MEIO



Como uma caixa postal: quem envia deixa a carta e vai embora; quem recebe pega quando passar por lá.

O broker guarda: se o consumidor está fora do ar, a mensagem espera. É isso que tira a corrente de falhas.

O broker absorve picos: chegaram 1.000 pedidos de uma vez? Eles esperam na fila, e o pagamento processa no ritmo dele.

Ninguém conhece ninguém: producer e consumers só conhecem o broker e o nome do tópico.

Exemplos de broker: Kafka, RabbitMQ, Amazon SQS/SNS, Google Pub/Sub, Azure Service Bus.

DOIS JEITOS DE ENTREGAR

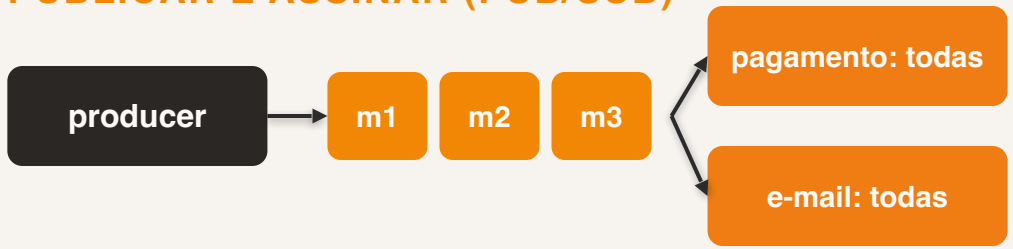
FILA (PONTO A PONTO)



Cada mensagem vai para UM consumidor.

Serve para dividir trabalho entre réplicas iguais.

PUBLICAR E ASSINAR (PUB/SUB)



Cada mensagem vai para TODOS os assinantes.

Serve para avisar vários serviços diferentes do mesmo fato.

No Kafka, os dois ao mesmo tempo, com consumer groups:

- **dentro de um grupo,** as mensagens são divididas (fila): as 3 réplicas do pagamento repartem o trabalho;
- **entre grupos diferentes,** cada grupo recebe tudo (pub/sub): pagamento e notificações leem o mesmo pedido.criado.

APACHE KAFKA

Criado no LinkedIn em 2011 para mover bilhões de eventos por dia entre sistemas. Hoje é projeto da Apache, de código aberto.

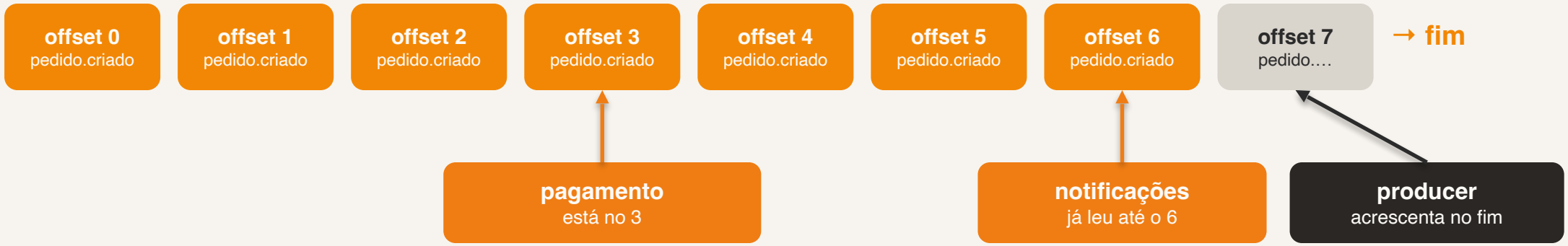
Usado para mensageria entre microserviços, rastreamento de cliques, logs, integração de dados e processamento em tempo real.

Netflix, Uber, LinkedIn e Nubank estão entre os que o usam em grande escala.

A IDEIA CENTRAL: um log que só cresce, que muitos podem ler.

A IDEIA CENTRAL: UM LOG

Um log é uma lista onde as mensagens só são acrescentadas no fim. Nada é alterado, nada é apagado ao ser lido.



Offset: a posição de cada mensagem. Começa em 0 e só cresce.

Cada leitor tem o seu marcador: o pagamento está no 3, as notificações no 6. Um não atrapalha o outro.

Ler não apaga: diferente de uma fila tradicional. As mensagens ficam pelo tempo de retenção (padrão: 7 dias).

Por isso dá para reler: voltar o marcador e processar de novo, ou um serviço novo ler o histórico desde o início.

AS PEÇAS DO KAFKA

Peça	O que é	Na docker-class
broker	um servidor Kafka; vários formam um cluster	o container kafka (um só)
tópico	um assunto, um nome para um log	pedidos, pagamentos
partição	um pedaço do tópico; o tópico é dividido para ganhar paralelismo	3 por tópico
offset	posição de uma mensagem na partição	0, 1, 2...
producer	quem publica no tópico	order-service, payment-service
consumer	quem lê do tópico	os três serviços
consumer group	consumers que dividem o trabalho e guardam o offset juntos	um grupo por serviço
chave (key)	define em qual partição a mensagem cai	o id do pedido

Mensagem = chave + valor (bytes; nós usamos JSON) + cabeçalhos + horário.

TÓPICO PEDIDOS, 3 PARTIÇÕES



Partição = um log independente, com os seus próprios offsets.

Por que dividir? Cada partição pode ser lida por um consumer diferente, ao mesmo tempo, e pode ficar num broker diferente.

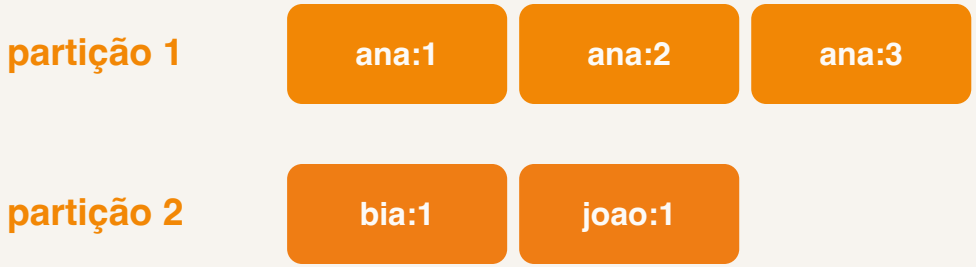
Escolhido na criação do tópico (nós: 3). Dá para aumentar depois, mas não diminuir, e aumentar muda para onde as chaves vão.

A partição é a unidade de: ordem (só dentro dela), paralelismo (um consumer por partição no grupo) e distribuição entre brokers.

Quantas? Pelo menos o número máximo de réplicas que um serviço vai ter. Poucas limitam a escala; muitas demais custam memória e arquivos.

A CHAVE DECIDE A PARTIÇÃO

```
partição = hash(chave) % partições
```



Resultado do passo 3 do roteiro.

Mesma chave → mesma partição → em ordem. Tudo da Ana chega na ordem em que foi publicado.

Chaves diferentes podem cair na mesma partição (bia e joao): o hash é que decide.

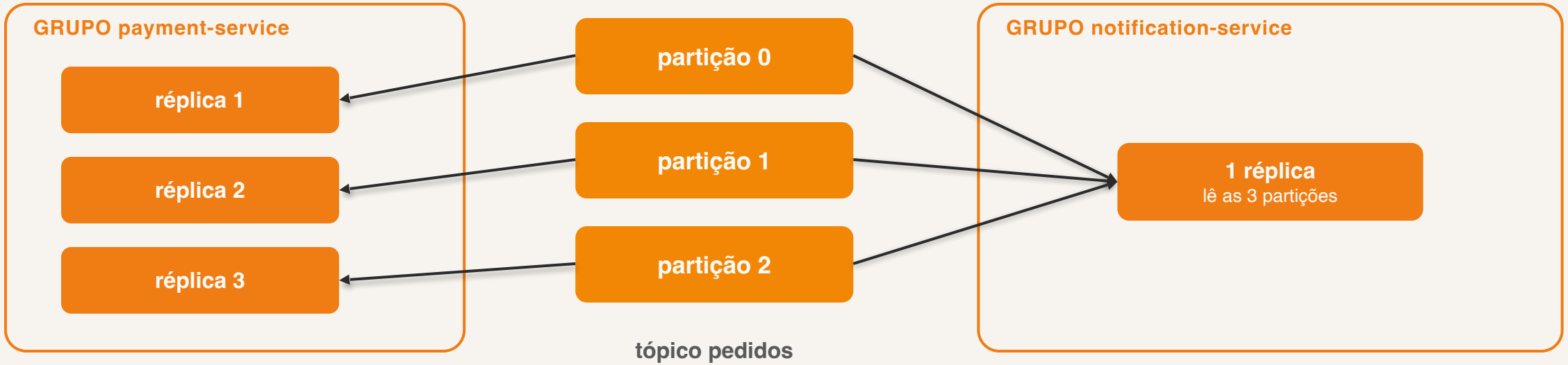
Sem chave: o producer espalha as mensagens; não há ordem garantida.

Ordem só dentro da partição. Entre partições, o Kafka não promete nada. Por isso a chave é o que precisa de ordem.

No projeto, a chave é o id do pedido: no tópico pedidos, o pedido.cancelado de um pedido nunca chega antes do pedido.criado dele. Pedidos diferentes, em paralelo.

Entre tópicos, não há ordem: estoque.reservado vem depois de pedido.criado porque o catálogo só o publica depois de ler o primeiro (causa e efeito).

CONSUMER GROUPS



Dentro do grupo: cada partição é de UMA réplica por vez. As 3 réplicas do pagamento repartem o trabalho.

Entre grupos: independentes. As notificações leem tudo, no seu ritmo, com o seu próprio offset.

O groupId é o nome do serviço: subir mais réplicas não muda nada no código, elas entram no mesmo grupo.

Rebalanceamento: uma réplica entrou ou saiu? O Kafka redistribui as partições. Enquanto isso, o grupo para por uns segundos.

RÉPLICAS × PARTIÇÕES

Réplicas no grupo	Com 3 partições	Resultado na docker-class
1	uma réplica lê as 3	9 pagamentos em ~20 s
3	uma partição para cada	9 pagamentos em ~8 s
4	três trabalham, uma fica com 0	a 4ª só espera alguém cair

O número de partições é o teto de réplicas trabalhando ao mesmo tempo num grupo.

A réplica parada não é inútil: se outra cair, ela assume a partição no rebalanceamento.

Escala é desigual se as chaves forem: muitos eventos com a mesma chave lotam uma partição só (partição quente).

O GRUPO LEMBRA ONDE PAROU



lido e confirmado (commit) **ainda não lido: LAG = 3**

```
GROUP            TOPIC    PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
payment-service  pedidos  0          5               8               3
```

Commit: o consumer avisa ao Kafka "processei até aqui". O Kafka guarda isso num tópico interno (__consumer_offsets).

CURRENT-OFFSET: de onde o grupo continua. LOG-END-OFFSET: o fim da partição.

LAG: quantas mensagens esperam. Lag crescendo = o consumidor não dá conta ou caiu. É a métrica mais vigiada de um sistema com Kafka.

Réplica nova ou reiniciada continua do commit do grupo, não do começo.

GARANTIAS DE ENTREGA

Garantia	Como acontece	Risco
No máximo uma vez	confirma (commit) ANTES de processar	caiu no meio: a mensagem se perde
Pelo menos uma vez	confirma DEPOIS de processar (padrão)	caiu antes de confirmar: processa de novo
Exatamente uma vez	transações do Kafka (de tópico para tópico)	não cobre o seu banco nem um e-mail enviado

Na prática: pelo menos uma vez. Perder um pedido é pior que receber o aviso duas vezes. Então o mesmo evento PODE chegar de novo:

- o serviço processou, gravou no banco e caiu antes de confirmar;
- houve rebalanceamento no meio do processamento;
- alguém voltou o offset de propósito para reprocessar.

Consequência: todo consumer precisa aguentar receber o mesmo evento duas vezes.

IDEMPOTÊNCIA

Processar duas vezes tem o mesmo efeito que processar uma. Três jeitos comuns:

CHAVE ÚNICA

A tabela não aceita o mesmo pedido duas vezes. O segundo INSERT não grava nada.

FILTRO DE ESTADO

Só muda se ainda estiver no estado esperado: WHERE status = 'AGUARDANDO_PAGAMENTO'.

REGISTRO DE EVENTOS

Guarda o id de cada evento processado e ignora os que já estão lá.

```
// payment-service da docker-class: pedido_id é a chave primária
const { rowCount } = await pool.query(
  'INSERT INTO pagamentos (pedido_id, valor, status) VALUES ($1,$2,$3) ON CONFLICT (pedido_id) DO NOTHING', ...);
if (rowCount === 0) return;  // já processado: ignora
```

No roteiro, passo 11: voltamos o offset para o início, os 23 pedidos foram relidos e nenhum pagamento se repetiu.

RETENÇÃO E REPROCESSAMENTO

RETENÇÃO

O Kafka guarda as mensagens por tempo (padrão: 7 dias) ou por tamanho, não até alguém ler.

Tópicos compactados guardam só a última mensagem de cada chave: o estado atual de cada pedido.

REPROCESSAR (REPLAY)

Voltar o offset de um grupo e ler de novo: corrigir um bug e reprocessar o que deu errado.

Serviço novo com fromBeginning: lê o histórico e monta o próprio banco a partir dos eventos.

```
kafka-consumer-groups.sh --group payment-service --topic pedidos \
  --reset-offsets --to-earliest --execute    # com o grupo parado
```

EM PRODUÇÃO: VÁRIOS BROKERS



Fator de replicação 3: cada partição tem 3 cópias, em brokers diferentes. Um broker cai, outra cópia vira líder e ninguém perde dados.

Líder: producers e consumers falam com o líder da partição; as cópias acompanham (ISR: in-sync replicas).

acks=all: o producer só considera publicado quando as cópias em dia confirmaram.

KRaft: desde o Kafka 3.3, o próprio Kafka coordena o cluster. Antes era preciso um ZooKeeper à parte.

Na docker-class e no projeto: um broker só, no modo KRaft, fator 1. Ótimo para aprender; inaceitável em produção.

KAFKA × RABBITMQ × SQS

	Kafka	RabbitMQ	Amazon SQS
Modelo	log distribuído	filas com roteamento	fila gerenciada
Depois de ler	a mensagem fica (retenção)	a mensagem sai da fila	a mensagem sai da fila
Reler o passado	sim, voltando o offset	não nas filas comuns (sim nos Streams)	não
Ordem	por partição	por fila	só na fila FIFO
Volume	milhões por segundo (cluster)	dezenas de milhares por nó	gerenciado pela AWS
Operar	mais complexo	mais simples	nada a operar
Bom para	eventos, histórico, muitos leitores	tarefas, roteamento flexível	fila simples na AWS

Por que Kafka no curso? É o mais usado em arquiteturas de eventos e mostra conceitos que os outros escondem: partição, offset, grupo, replay.

AO VIVO: KAFKA NA DOCKER-CLASS

Os mesmos order e payment, agora conversando por eventos, com um serviço de notificações escutando tudo.

1. Criar um tópico com 3 partições

2–3. Producer e consumer na mão: chave e ordem

4. Mensagem em tempo real

5. Consumer group e LAG

6. Mesmo grupo divide; outro grupo recebe tudo

7. Pedido pelo Kong: 202 e depois PAGO

8. Kafka UI: envelope e correlationId

9. Pagamento fora do ar: o evento espera

10. Escalar o consumidor: 3 e 4 réplicas

11. Reprocessar: idempotência

ROTEIRO: `docker-class/ROTEIRO-KAFKA.md`

DESENHANDO EVENTOS: NOMES

ASSIM	NÃO ASSIM
pedido.criado	criarPedido
estoque.reservado	reservarEstoque
estoque.insuficiente	ProcessarPagamento
pagamento.aprovado	atualizacao

entidade.fato, no passado: algo que já aconteceu. Imperativo vira comando disfarçado; nome genérico ninguém entende.

Tópicos por entidade: pedidos, estoque, pagamentos. Cada tópico leva os eventos daquela entidade, e o campo tipo diz qual é.

Quem publica é o dono do fato: só o catálogo publica estoque.*; só pedidos publica pedido.*.

O ENVELOPE: TODO EVENTO IGUAL POR FORA

```
{
  "id": "6f1c...-uuid",
  "tipo": "pedido.criado",
  "pedidoId": "2",
  "ocorridoEm": "2026-10-26T19:51:04Z",
  "correlationId": "34781360-...",
  "dados": {
    "usuarioId": "ana",
    "valorTotal": 120
  }
}
```

Mesmo envelope em todos os serviços: um kafka.js igual em cada um publica e lê sempre assim.

`id:` único por evento: permite detectar repetição.

`tipo:` o consumer decide o que fazer por ele; tipos que não conhece, ignora.

`pedidoId:` também é a chave da mensagem (ordem por pedido).

`ocorridoEm:` quando o fato aconteceu, não quando foi lido.

`correlationId:` o X-Request-Id do Kong. Segue de evento em evento: um grep mostra a compra inteira.

`dados:` o conteúdo, diferente para cada tipo.

QUANTO COLOCAR DENTRO DO EVENTO?

MAGRO (SÓ O AVISO)

{ tipo: pedido.criado, pedidoId: 2 }

Quem recebe precisa chamar o pedidos para saber o resto. Volta o acoplamento: pedidos fora, ninguém processa.

COM O NECESSÁRIO

{ ..., dados: { itens, valorTotal, clienteEmail } }

Quem recebe já tem tudo para trabalhar. Ninguém precisa consultar ninguém.

No projeto: estoque.reservado repassa valorTotal, formaPagamento e clienteEmail que vieram no pedido.criado. Na próxima aula, o pagamento cobra só com o evento.

Mas sem exagero: número de cartão, senha e documento não vão para o evento. Ele fica guardado e muita gente lê.

EVENTO É CONTRATO

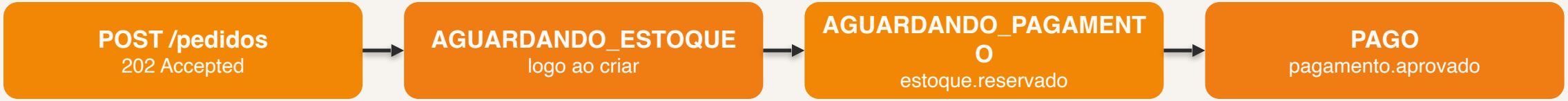
Mudança no evento	Efeito em quem já escuta
Acrescentar um campo	nada: quem não conhece, ignora
Renomear um campo	quebra: o consumer procura o nome antigo
Remover um campo	quebra quem usava
Mudar o tipo (texto → número)	quebra em silêncio
Mudar o significado	o pior: ninguém percebe

Regra: só acrescente. Precisa quebrar? Publique um tipo novo (pedido.criado.v2) por um tempo, junto com o antigo.

Lembre do replay: eventos antigos continuam no tópico. O consumer precisa entender os formatos velhos também.

Em empresas grandes: um Schema Registry guarda o formato de cada evento e recusa mudanças que quebram.

CONSISTÊNCIA EVENTUAL



tempo → o mesmo GET /pedidos/:id devolve respostas diferentes em instantes diferentes

O sistema fica coerente "daqui a pouco", não no instante da resposta. Por alguns segundos, o pedido existe e o estoque ainda não foi reservado.

202 Accepted: o jeito HTTP de dizer "recebi, ainda estou trabalhando". A resposta já traz o id e o status atual.

Como o cliente fica sabendo: consulta de novo (polling), recebe um e-mail, ou o servidor avisa (WebSocket).

O status é a verdade: o front mostra "processando" enquanto o pedido não chega a PAGO ou CANCELADO.

Isso assusta, mas é o normal: o banco mostra o PIX "em processamento"; a loja manda "pedido recebido" antes de "pagamento aprovado".

QUEM CONDUZ O FLUXO?

COREOGRAFIA

Ninguém manda: cada serviço reage aos eventos e publica os seus.

Pedidos não sabe que o catálogo reserva; só publica pedido.criado.

Simples para começar; o fluxo inteiro só aparece somando todos.

ORQUESTRAÇÃO

Um coordenador diz a cada serviço o que fazer, em ordem, por comandos.

O fluxo fica num lugar só, fácil de ler.

O coordenador conhece todos: mais acoplamento.

No projeto: coreografia. E quando um passo falha no meio (estoque reservado, pagamento recusado)? Desfazer os passos anteriores com eventos é a SAGA: assunto da próxima aula.

AS ARMADILHAS

Problema	O que acontece	Como se trata
Escrita dupla	grava o pedido, o Kafka cai antes de publicar: pedido parado para sempre	padrão Outbox: grava o evento numa tabela na mesma transação
Evento repetido	pelo menos uma vez: o mesmo pedido.criado chega duas vezes	idempotência: chave única, filtro de estado
Mensagem venenosa	um evento com defeito dá erro sempre; o consumer trava nele	tentar N vezes e mandar para um tópico de erros (DLQ)
Ordem	eventos do mesmo pedido em partições diferentes chegam fora de ordem	chave = id do pedido
Depurar	o erro está em qual dos 5 serviços?	correlationId em todos os logs; Kafka UI; lag

Na referência: idempotência, chave e correlationId estão prontos. Outbox e DLQ ficam como limitações conhecidas, descritas no README.

KAFKAJS: PUBLICAR

```
const { Kafka } = require('kafkajs');

const kafka = new Kafka({
  clientId: 'pedidos',
  brokers: ['kafka:9092'],
});
const producer = kafka.producer();
await producer.connect();

await producer.send({
  topic: 'pedidos',
  messages: [{
    key: pedido.id,                  // ordem por pedido
    value: JSON.stringify(evento),
  }],
});
```

Conectar uma vez quando o serviço sobe, não a cada envio.

`value` é texto: o Kafka não sabe o que é JSON. Quem lê faz o JSON.parse.

Publicar depois do COMMIT no banco: nunca anuncie um pedido que pode sumir num rollback.

Qualquer linguagem: Java, Python, Go e C# têm clientes oficiais ou maduros. O tópico não sabe quem publicou.

KAFKAJS: CONSUMIR

```
const consumer = kafka.consumer({ groupId: 'catalogo' });
await consumer.connect();
await consumer.subscribe({ topic: 'pedidos', fromBeginning: true });

await consumer.run({
  eachMessage: async ({ message }) => {
    const evento = JSON.parse(message.value.toString());
    const tratar = handlers[evento.tipo];
    if (!tratar) return;          // tipo que não me interessa
    await tratar(evento);                 // terminou: commit
  },
});

// ao parar o container: sai do grupo na hora
process.once('SIGTERM', () => consumer.disconnect());
```

groupId = nome do serviço: réplicas dividem as partições sozinhas.

`fromBeginning` só vale para um grupo novo, sem offset guardado.

O kafkajs confirma o offset depois que o eachMessage termina: pelo menos uma vez.

Desligar com calma: sem o disconnect, o Kafka espera ~30 s para repassar as partições. No compose: stop_grace_period: 10s.

O KAFKA NO DOCKER-COMPOSE.YML

```yaml
kafka:
  image: apache/kafka:3.8.0
  environment:
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller    # KRaft
    KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,...
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_NUM_PARTITIONS: 3
  healthcheck:
    test: ["CMD-SHELL", "kafka-topics.sh ... --list"]

kafka-ui:
  image: kafbat/kafka-ui:v1.1.0
  ports: ["8090:8080"]
```

Um container faz tudo: broker e controller (KRaft), sem ZooKeeper.

ADVERTISED_LISTENERS: o endereço que o Kafka informa aos clientes: kafka:9092, o nome na rede do compose.

Fator 1: só existe um broker para guardar cópias.

Serviços esperam o Kafka: depends_on com condition: service_healthy.

O Kafka não publica porta: só os serviços da rede interna falam com ele.

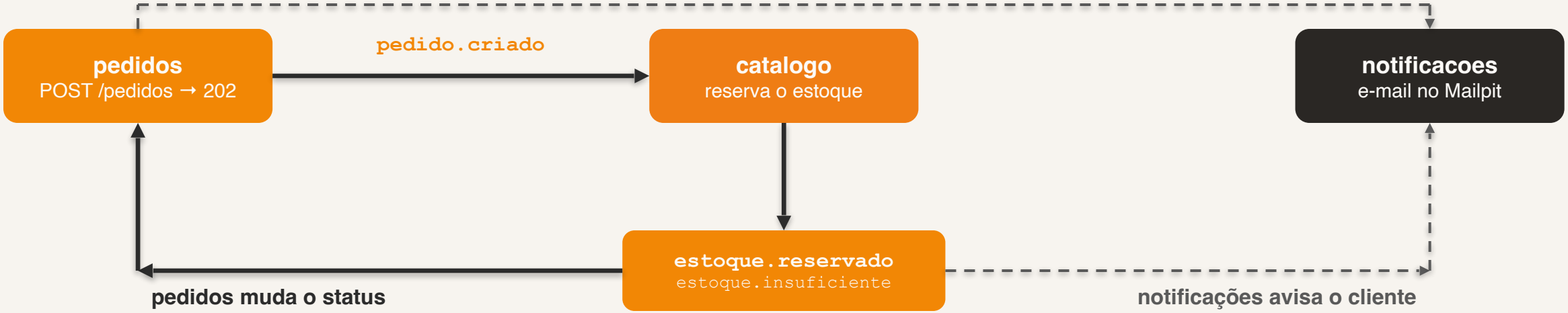
NO PROJETO: O CHECKOUT VIRA EVENTO

Na aula passada, o checkout respondia 202 e o pedido ficava parado em AGUARDANDO_ESTOQUE.

Hoje ele anda sozinho: o catálogo reserva o estoque, o pedido muda de status e o cliente recebe e-mail. O pagamento entra na próxima aula.

ENTRA NO COMPOSE: kafka, kafka-ui, notificacoes e mailpit.

O FLUXO DE HOJE NO E-COMMERCE



1. Checkout: pedidos grava o pedido em AGUARDANDO_ESTOQUE, publica pedido.criado (tópico pedidos) e responde 202.

2. Catálogo reserva tudo ou nada e publica estoque.reservado ou estoque.insuficiente (tópico estoque).

3. Pedidos escuta o tópico estoque: AGUARDANDO_PAGAMENTO ou CANCELADO, com o motivo.

4. Notificações escuta os dois tópicos e manda e-mail: "recebemos o seu pedido" e "sem estoque".

Linha cheia: quem reage e muda dados. Tracejada: quem só escuta para avisar.

TÓPICOS E EVENTOS

Evento	Tópico	Publica	Consome	dados
pedido.criado	pedidos	pedidos	catalogo, notificacoes	usuarioId, clienteEmail, itens, valorTotal, formaPagamento
estoque.reservado	estoque	catalogo	pedidos (e pagamentos, na próxima aula)	usuarioId, clienteEmail, valorTotal, formaPagamento
estoque.insuficiente	estoque	catalogo	pedidos, notificacoes	motivo, usuarioId, clienteEmail

Chave de todas as mensagens: o id do pedido. Envelope: { id, tipo, pedidoId, ocorridoEm, correlationId, dados }.

Os tópicos são criados por quem sobe primeiro (listTopics + createTopics), com 3 partições.

Group ids: pedidos, catalogo, notificacoes: um por serviço.

Próxima aula: tópico pagamentos, com pagamento.aprovado e pagamento.recusado.

PEDIDOS: PUBLICAR E ESCUTAR

```
// checkout: depois do COMMIT
await publicar('pedidos', 'pedido.criado', pedido.id, {
  usuarioId, clienteEmail, valorTotal,
  formaPagamento, cartaoFinal, itens,
}, req.requestId);
res.status(202).json(pedido);

// reações aos eventos do estoque
const handlers = {
  'estoque.reservado': e => mudarStatus(e.pedidoId,
      'AGUARDANDO_PAGAMENTO', ['AGUARDANDO_ESTOQUE']),
  'estoque.insuficiente': e => mudarStatus(e.pedidoId,
      'CANCELADO', ['AGUARDANDO_ESTOQUE'], e.dados.motivo),
};
consumir(['estoque'], handlers);

// mudarStatus: UPDATE ... SET status = $2
//   WHERE id = $1 AND status = ANY($3)
```

Publica depois do commit: nunca anuncie um pedido que pode sumir num rollback.

O evento leva o que os outros precisam: itens para o catálogo, e-mail para as notificações.

Filtro de estado: só muda se estiver no status esperado. estoque.reservado repetido não faz nada.

A consulta síncrona ao catálogo continua: o preço precisa ser o de agora, no checkout.

CATALOGO: RESERVAR SEM REPETIR

```
// 1. idempotência: reservas.pedidoId é único
try { await db.reservas.insertOne({ pedidoId, itens,
        status: 'processando' }); }
catch (e) { if (e.code === 11000) return; } // já veio

// 2. tudo ou nada: só tira se houver
for (const item of dados.itens) {
  const r = await db.produtos.updateOne(
    { _id: item.produtoId,
      'estoque.disponivel': { $gte: item.quantidade } },
    { $inc: { 'estoque.disponivel': -qtd,
              'estoque.reservado':  qtd } });
  if (!r.modifiedCount) {   // faltou: devolve o que já tirou
    // ... publicar estoque.insuficiente; return
  }
}
// 3. deu tudo certo
publicar('estoque', 'estoque.reservado', pedidoId, {...});
```

Coleção reservas: uma por pedido, com os itens. Índice único em pedidoId.

$gte no filtro: o Mongo só tira se houver; dois pedidos ao mesmo tempo não deixam o estoque negativo.

Tudo ou nada: faltou um item, devolve os que já reservou.

Reservado ≠ vendido: disponivel diminui, reservado aumenta. Na próxima aula, o pagamento confirma ou devolve.

NOTIFICACOES + MAILPIT

```
const smtp = nodemailer.createTransport({
  host: 'mailpit', port: 1025, secure: false,
});

// para cada evento que interessa:
await smtp.sendMail({
  to: evento.dados.clienteEmail,
  subject: `Recebemos o seu pedido`,
  text: `...`,
});
```

```
  mailpit:
    image: axllent/mailpit
    ports: ["8025:8025"]    # caixa de entrada
  notificacoes:
    build: ./notificacoes     # sem banco, sem rota
```

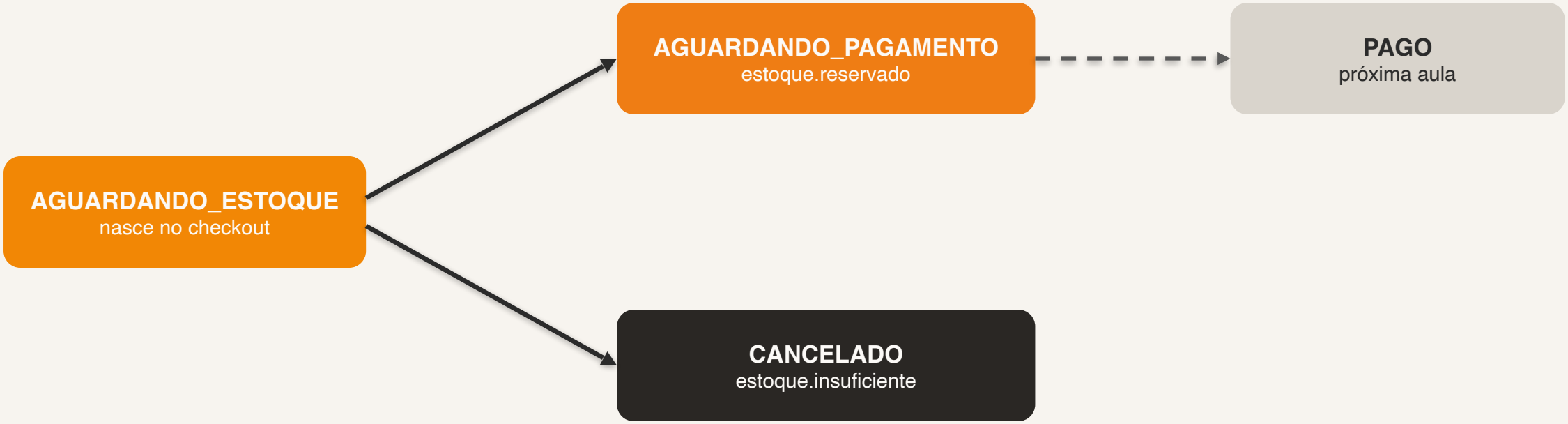
Mailpit: um servidor de e-mail de mentira. Recebe tudo e mostra numa caixa de entrada em localhost:8025. Nada sai para a internet.

Serviço só de escuta: ninguém chama, ninguém conhece. Se cair, a loja continua; os eventos esperam.

Por isso o clienteEmail vai dentro do evento: o serviço não precisa consultar o de usuários.

No Kong: nenhuma rota. Ele não tem API.

O STATUS DO PEDIDO HOJE



Cada seta é um evento, e cada mudança só vale a partir do estado de origem: um evento repetido ou atrasado não leva o pedido para trás.

POST /pedidos/:id/cancelar continua permitido em AGUARDANDO_ESTOQUE e AGUARDANDO_PAGAMENTO. Devolver o estoque do cancelado fica para a saga, na próxima aula.

TESTANDO O FLUXO

```
### Checkout: 202 e AGUARDANDO_ESTOQUE
# @name checkout
POST {{baseUrl}}/pedidos
Authorization: Bearer {{login.response.body.$.token}}
Content-Type: application/json

{ "enderecoEntrega": {...}, "formaPagamento": "pix" }

### Reenvie até mudar: AGUARDANDO_PAGAMENTO
GET {{baseUrl}}/pedidos/{{checkout.response.body.$.id}}
Authorization: Bearer {{login.response.body.$.token}}
```

Caminho feliz: 202, depois AGUARDANDO_PAGAMENTO; o estoque.disponivel do produto diminuiu e o reservado aumentou.

Sem estoque: 3 unidades do Celular Básico (o seed tem 2): CANCELADO, com o motivo.

Kafka UI (8090): os eventos nos tópicos pedidos e estoque, com a mesma chave.

Mailpit (8025): os dois e-mails.

Logs: `docker compose logs | grep <correlationId>`

PARA DISCUTIR EM DUPLA

1. O checkout ainda consulta o catálogo de forma síncrona. Dá para tirar essa chamada também? O que se perde?
2. O Kafka caiu entre o COMMIT do pedido e o publicar. O que acontece com esse pedido?
3. Por que a chave das mensagens é o id do pedido e não o id do usuário?
4. O catálogo tem 3 réplicas e o tópico pedidos, 3 partições. E se subirmos 5 réplicas?
5. O cliente cancelou um pedido que já estava com o estoque reservado. Quem devolve o estoque?

O QUE ENTREGAR

1. Kafka (KRaft) e Kafka UI no compose; **serviços esperam o Kafka ficar healthy.**
2. **pedidos publica pedido.criado** depois do commit, com o envelope e a chave = id do pedido.
3. **catalogo reserva** tudo ou nada, sem repetir (reservas.pedidoId único), e publica estoque.*.
4. **pedidos escuta estoque.*** e muda o status só a partir de AGUARDANDO_ESTOQUE.
5. **notificacoes + Mailpit:** e-mail de pedido recebido e de estoque insuficiente.
6. **correlationId** (X-Request-Id do Kong) em todos os eventos e logs.
7. **http/pedidos.http** com o caminho feliz e o sem estoque; README com a tabela de eventos.

CHECKPOINT: tag `aula-kafka` no repositório.

PRÓXIMA AULA

01 Entra o serviço de pagamentos, escutando estoque.reservado.

02 Um banco externo de mentira que falha e demora de propósito.

03 Saga: pagamento recusado devolve o estoque; cancelamento estorna.

04 Resiliência: timeout, retry e circuit breaker.

Contato



Augusto Gehrke

https://www.linkedin.com/in/augusto-g-b54a5bb7

augusto.gehrke@amf.edu.br

augustogehrke@outlook.com